

RandomDataStreams.jl

August 26, 2026

Contents

Contents	ii
I Home	1
1 RandomDataStreams.jl Documentation	2
1.1 Contents	2
1.2 Quick example	2
II Getting Started	3
2 Getting Started	4
2.1 Installation	4
2.2 Choosing a generator	4
2.3 First numbers	5
2.4 Supported output types	5
2.5 Reproducibility	5
2.6 Full Random API	6
2.7 Next steps	6
III Streams & Substreams	7
3 Streams & Substreams	8
3.1 Why streams? The case for Common Random Numbers (CRN)	8
3.2 The two object families	10
3.3 Producing independent streams	10
3.4 Navigating substreams	10
3.5 Saving, restoring, jumping	11
3.6 Guarantees	11
IV API Reference	12
4 API Reference	13
4.1 Types	13
4.2 MRG32k3a	13
4.3 Xoshiro256+	14
4.4 Xoshiro / xoroshiro families	15
4.5 Method index	15
V Docstrings	17
5 Docstrings	18
5.1 Abstract types	18
5.2 MRG32k3a	18
5.3 Xoshiro / xoroshiro families	19

VI	Implementation Notes	26
6	Implementation Notes	27
6.1	MRG32k3a	27
6.2	Xoshiro256+	28
6.3	Integer outputs of MRG32k3a	29
6.4	Testing strategy	29
6.5	Performance snapshot	29
VII	Generator Comparison	30
7	Comparing generators: xoshiro variants vs MRG32k3a	31
7.1	Side-by-side	31
7.2	When to choose MRG32k3a	32
7.3	When to choose a xoshiro/xoroshiro variant	32
7.4	Scrambler choice within a family	32
7.5	Practical notes	32
VIII	FAQ	33
8	FAQ	34
8.1	Why is an all-zero state forbidden?	34
8.2	Are MRG32k3a integer outputs uniform over their full width?	34
8.3	Why doesn't <code>sample(v, k)</code> work with <code>RandomDataStreams</code> generators?	34
8.4	Where did <code>srand</code> , <code>next_stream</code> , <code>short_jump</code> , <code>long_jump</code> go?	34
8.5	How do I run truly parallel simulations?	34
8.6	Which generator should I pick?	35

Part I

Home

Chapter 1

RandomDataStreams.jl Documentation

Streamable pseudo-random number generators for Julia: MRG32k3a (L'Ecuyer) and xoshiro256+ (Blackman & Vigna), with non-overlapping streams and substreams.

1.1 Contents

1. [Getting Started](#)
2. [Streams & Substreams](#)
3. [API Reference](#)
4. [Implementation Notes](#)
5. [Generator Comparison](#)

1.2 Quick example

```
using RandomDataStreams

gen = MRG32k3aGen()
rngA = next_stream!(gen)    # independent stream A
rngB = next_stream!(gen)    # independent stream B

rand(rngA)                  # Float64 in [0, 1)
next_substream!(rngA)       # move to the next substream of A
reset_stream!(rngA)         # rewind stream A to its beginning
```

Part II

Getting Started

Chapter 2

Getting Started

RandomDataStreams.jl provides pseudo-random number generators with support for **non-overlapping streams and substreams**, following the stream/substream model of L'Ecuyer et al. (2002).

2.1 Installation

```
using Pkg
Pkg.add(url = "https://github.com/wirelessroom2/RandomDataStreams.jl")
```

or, from a local clone:

```
using Pkg
Pkg.develop(path = "path/to/RandomDataStreams.jl")
```

Requirements: Julia ≥ 1.6 . The only dependency is the standard library Random.

2.2 Choosing a generator

RandomDataStreams.jl implements the full xoshiro/xoroshiro family of Blackman & Vigna plus L'Ecuyer's MRG32k3a. All variants of a xoshiro family share the same linear transition and jump constants; only the output scrambler differs.

Type	Family	State	Period	Scrambler
Xoroshiro128p	xoroshiro128	128 bits	$2^{128} - 1$	$s_0 + s_1$
Xoroshiro128ss	xoroshiro128	128 bits	$2^{128} - 1$	high bits, **
Xoroshiro128pp	xoroshiro128	128 bits	$2^{128} - 1$	high bits, ++
Xoshiro256p	xoshiro256	256 bits	$2^{256} - 1$	$s_0 + s_3$
Xoshiro256ss	xoshiro256	256 bits	$2^{256} - 1$	high bits, **
Xoshiro256pp	xoshiro256	256 bits	$2^{256} - 1$	high bits, ++
Xoshiro512p	xoshiro512	512 bits	$2^{512} - 1$	$s_0 + s_2$
Xoshiro512ss	xoshiro512	512 bits	$2^{512} - 1$	high bits, **
Xoshiro512pp	xoshiro512	512 bits	$2^{512} - 1$	high bits, ++
MRG32k3a	combined MRG (L'Ecuyer)	192 bits	$\approx 2^{191}$	native Float64

See [Generator Comparison](#) for guidance and benchmarks.

2.3 First numbers

```
using RandomDataStreams

rng = MRG32k3a()      # default seed [12345, ..., 12345]
rand(rng)             # 0.12701112204657714
rand(rng)             # 0.3185275653967945
```

```
x = Xoshiro256p(UInt64[1, 2, 3, 4])
rand(x)               # Float64 in [0, 1)
```

2.4 Supported output types

MRG32k3a

- Float64 (native), plus Float32, Float16 via conversion
- UInt8, UInt16, UInt32, UInt64, UInt128
- Int8, Int16, Int32, Int64, Int128 (same bit patterns)
- Bool
- Ranges: `rand(rng, 1:10)` works through the standard Random machinery

Note: integer outputs of MRG32k3a are built from its native ~31-bit resolution; they are not uniform over their full width. Use them for indices, choices and flags rather than cryptography.

Xoshiro256p

- Float64 (native path), Float32, Float16
- UInt64
- Ranges: `rand(rng, r::UnitRange{Int64})`

2.5 Reproducibility

Every generator accepts an explicit seed:

```
rng = MRG32k3a([42, 1, 2, 3, 4, 5])
x = Xoshiro256p(UInt64[0xdead, 0xbeef, 0xcafe, 0xbabe])
```

Seeds are validated by `checkseed`: a valid MRG32k3a seed has 6 non-negative values; the first three must be $< m1$ and not all zero, the last three $< m2$ and not all zero.

The standard Julia interface also works with every generator:


```
Random.seed!(rng, 42)           # integer seed (splitmix64 expansion)
Random.seed!(rng, [7,7,7,8,8]) # explicit MRG32k3a seed (validated)
```

2.6 Full Random API

RandomDataStreams generators are drop-in substitutes for Julia's built-in RNGs: anywhere a function accepts an AbstractRNG, you can pass an RandomDataStreams generator and keep your stream/substream control.

```
using RandomDataStreams, Random

rng = next_stream!(MRG32k3aGen()) # any RandomDataStreams generator works here

rand(rng, 5)                      # Vector{Float64}, 5 draws
A = rand(rng, Float64, 2, 3)      # 2x3 matrix
z = randn(rng)                   # standard normal
e = randexp(rng)                 # exponential
v = shuffle(rng, collect(1:8))   # shuffled copy
p = randperm(rng, 6)             # random permutation
buf = Vector{Float64}(undef, 3)
rand!(rng, buf)                  # fill in place
Random.randstring(rng, 10)       # random string
```

Seeding through the standard interface makes results reproducible:

```
Random.seed!(rng, 42)
u1 = [rand(rng) for _ in 1:3]
Random.seed!(rng, 42)
u2 = [rand(rng) for _ in 1:3]
u1 == u2                        # true
```

Only sample requires StatsBase.jl and is out of scope.

2.7 Next steps

- [Streams & substreams](#)
- [API reference](#)
- [Implementation notes](#)

Part III

Streams & Substreams

Chapter 3

Streams & Substreams

The central abstraction of RandomDataStreams.jl is the *stream*: a long, non-overlapping subsequence of a generator's period. Streams can themselves be split into *substreams*. This is the architecture recommended by L'Ecuyer et al. (2002) for parallel and replicated stochastic simulation:

- **Streams** are handed to independent replications or parallel workers; their outputs never overlap, so replications are statistically independent.
- **Substreams** delimit scenarios *within* one replication; rewinding a substream while keeping the scenario structure enables techniques such as common random numbers.

3.1 Why streams? The case for Common Random Numbers (CRN)

The whole point of the stream/substream architecture is **variance reduction when comparing systems** — arguably the most important technique in stochastic simulation design (L'Ecuyer et al. 2002; Law, *Simulation Modeling and Analysis*).

Suppose you compare K configurations of a stochastic system (two inventory levels, two queue disciplines...). The naive approach runs each configuration with *independent* random numbers. The variance of the estimated difference is then

$$\text{Var}[\hat{D}] = \text{Var}[\bar{Y}_1]/n + \text{Var}[\bar{Y}_2]/n$$

With **common random numbers**, every configuration of a given replication consumes the *same* underlying uniforms: the per-replication differences are strongly positively correlated and

$$\text{Var}[\hat{D}_{\text{CRN}}] = \text{Var}[\bar{Y}_1]/n + \text{Var}[\bar{Y}_2]/n - 2 \cdot \text{Cov}[\bar{Y}_1, \bar{Y}_2]/n$$

— often orders of magnitude smaller. But CRN only works if the random number usage can be *synchronised exactly* across configurations, replication after replication. That is precisely what RandomDataStreams's machinery guarantees:

Measurable example: comparing two inventory levels

Daily cost of an inventory system over 30 days (demands $N(100, 20)$ truncated at 0), comparing stock level $s = 50$ vs $s = 60$:

Need	RandomDataStreams tool
Replications must be independent	a fresh stream per replication (<code>next_stream!</code>)
Configurations must see identical randomness	rewind each configuration to the same substream start (<code>reset_stream!</code> / <code>reset_substream!</code>)
No accidental overlap between replications	streams are provably disjoint

```

using RandomDataStreams, Statistics

function cost(s, rng)
    total, stock = 0.0, Float64(s)
    for _ in 1:30
        stock = min(stock + 40, 200)
        d = max(0.0, 100 + 20randn(rng))
        sold = min(stock, d)
        total += 5(d - sold) + 0.1max(0, stock - sold - 10)
        stock -= sold
    end
    total
end

function replications(crn::Bool, n::Int)
    gen = MRG32k3aGen()
    diffs = Vector{Float64}{}(undef, n)
    for i in 1:n
        r50 = next_stream!(gen)
        l50 = cost(50, r50)
        r60 = crn ? reset_stream!(copy(r50)) : next_stream!(gen)
        l60 = cost(60, r60)
        diffs[i] = l60 - l50
    end
    return diffs
end

for crn in (true, false)
    ds = replications(crn, 4_000)
    println("CRN = \${crn}: mean diff ≈ \$(round(mean(ds), digits=1)), ",
           "var(diff) ≈ \$(round(var(ds), digits=1))")
end

```

Typical output:

```

CRN = true: mean diff ≈ -49.9, var(diff) ≈ 0.1
CRN = false: mean diff ≈ -39.2, var(diff) ≈ 540775.9

```

The variance of the comparison drops by **six orders of magnitude**: with CRN, both policies face identical demand sequences, so the observed difference measures the effect of the policy change rather than the noise of unrelated demand scenarios. Without CRN you would need $\sim 10^6$ more replications for the same precision.

This example uses `copy(r50) + reset_stream!`: both policies replay the very same stream from its beginning. `reset_substream!` generalises the pattern when each replication itself contains several scenarios.

3.2 The two object families

1. **Stream generators** (`<: AbstractRNGStream`) hold the seed of the *next* stream to be produced:
 - MRG32k3aGen
 - Xoshiro256plusGen
2. **RNGs** (`<: AbstractStreamableRNG`) actually produce numbers and can move along streams/substreams:
 - MRG32k3a
 - Xoshiro256p

3.3 Producing independent streams

```
using RandomDataStreams

gen = MRG32k3aGen()           # or Xoshiro256plusGen(UInt64{...})

worker1 = next_stream!(gen)    # stream 1
worker2 = next_stream!(gen)    # stream 2 – guaranteed disjoint from stream 1
worker3 = next_stream!(gen)    # stream 3 – ...
```

Each call to `next_stream!` advances the generator's internal seed by a huge leap (2^{127} values for MRG32k3a, a full `long_jump!` for Xoshiro256+), which is what guarantees non-overlap.

```
# Multithreaded simulation: one stream per thread.
# IMPORTANT: the generator object itself is not thread-safe – never share it;
# ship each worker its own stream *starting seed* instead.
using RandomDataStreams, Base.Threads

gen = MRG32k3aGen()
starts = Vector{Vector{Int}}(undef, nthreads())
for t in 1:nthreads()
    starts[t] = copy(get_state(gen)) # seed of stream t
    next_stream!(gen)                # advance to the following stream
end

results = Vector{Float64}(undef, nthreads())
@threads for t in 1:nthreads()
    s = starts[t]
    rng = MRG32k3a(s, s, s)          # rebuild stream t on this thread
    results[t] = sum(rand(rng) for _ in 1:10^5)
end
```

The same pattern works with `Distributed`: compute the list of starting seeds on the master process and send one entry per worker (`@spawnat` / `remotecall`), then rebuild the generator locally with the triple constructor.

3.4 Navigating substreams

Within a single RNG, three functions move along the stream/substream hierarchy (all return the modified RNG):

Function	Effect
<code>reset_stream!(rng)</code>	jump to the very beginning of the current stream
<code>reset_substream!(rng)</code>	jump to the beginning of the current substream
<code>next_substream!(rng)</code>	advance to the next substream

Example — common random numbers across two scenarios:

```
rng = next_stream!(MRG32k3aGen())

rand(rng)           # consume some variates of substream 1...
rand(rng)

next_substream!(rng) # switch to substream 2 for scenario B
uB = rand(rng)

reset_stream!(rng)   # replay substream 1 from scratch for scenario A
uA = rand(rng)       # same underlying uniforms as the first draw
```

For Xoshiro256+, `next_substream!` is implemented as a `short_jump!` (2^{96} -value leap) and `reset_stream! / next_substream!` manipulate the saved Bg/Ig checkpoints.

3.5 Saving, restoring, jumping

```
state = get_state(rng)   # a copy; safe to store
# ... consume numbers ...
```

To restore an MRG32k3a state, rebuild the generator with the triple constructor (Cg, Bg, Ig all set to the snapshot):

```
snapshot = get_state(rng)
xs = [rand(rng) for _ in 1:5]
clone = MRG32k3a(snapshot, snapshot, snapshot)
rand(clone) == xs[1]     # true - resumes exactly after the snapshot
```

MRG32k3a additionally supports arbitrary jumps inside a stream:

```
advance_state!(rng, e, c)
```

moves the state forward by n steps, where $n = 2^e + c$ (e may be negative, and negative c moves backwards). This costs $O(\log n)$ matrix operations, independent of the distance jumped.

3.6 Guarantees

- Streams produced by successive calls to `next_stream!` on the same generator object are **provably non-overlapping**.
- Substreams likewise partition a stream into disjoint blocks (length $\approx 2^{76}$ steps for MRG32k3a).

Part IV

API Reference

Chapter 4

API Reference

All names below are exported by the `RandomDataStreams` module unless marked (*internal*).

4.1 Types

AbstractRNGStream — supertype of stream *generators* (objects that mint new independent streams): `MRG32k3aGen`, `Xoshiro256plusGen`.

AbstractStreamableRNG <: `Random.AbstractRNG` — supertype of usable RNGs that can navigate streams and substreams: `MRG32k3a`, `Xoshiro256p`.

4.2 MRG32k3a

Constructors

Signature	Description
<code>MRG32k3a()</code>	default seed [12345, 12345, 12345, 12345, 12345, 12345]
<code>MRG32k3a(x::Vector{Int})</code>	seed all three states (Cg = Bg = Ig = x)
<code>MRG32k3a(x, y, z::Vector{Int})</code>	explicit current/substream/stream starts

Seeds are validated with `checkseed`; invalid seeds throw an `AssertionError`.

Functions

rand(rng::MRG32k3a) -> Float64 Uniform in [0, 1) with ~32 bits of resolution. This is the native output.

rand(rng::MRG32k3a, T) Supported T: `Float64`, `Float32`, `Float16`, `UInt8...UInt128`, `Int8...Int128`, `Bool`, and any `UnitRange` (e.g. `rand(rng, 1:10)`).

reset_stream!(rng) -> rng Set the current state (Cg) *and* the substream start (Bg) back to the stream start (Ig).

reset_substream!(rng) -> rng Set Cg back to Bg.

next_substream!(rng) -> rng Advance Bg to the next substream boundary (jump of $\approx 2^{76}$ steps) and set Cg = Bg.

advance_state!(rng, e::Integer, c::Integer) Jump forward/backward inside the current substream by n steps where $n = 2^e + c$. Negative e or c move backwards. Cost is $O(\max(|e|, \log |c|))$ matrix operations.

get_state(rng) -> Vector{Int} Return a copy of Cg.

checkseed(x) -> Bool true iff x has length 6, all entries ≥ 0 , entries 1–3 are $< m_1$ and not all zero, entries 4–6 are $< m_2$ and not all zero.

DEFAULT_SEED The constant seed vector used by MRG32k3a() and MRG32k3aGen().

copy(m::MRG32k3a) -> MRG32k3a Independent deep copy of the generator (all three state vectors).

Stream generation

Signature	Description
MRG32k3aGen(), MRG32k3aGen(seed)	create a stream generator
next_stream!(gen) -> MRG32k3a	return a new stream; internal seed leaps 2^{127} values ahead
get_state(gen) -> Vector{Int}	seed that will be used by the next next_stream! call

4.3 Xoshiro256+

Constructors

Signature	Description
Xoshiro256p(s::NTuple{4,UInt64}) / Xoshiro256p(v::Vector{<:Unsigned})	Cg = Bg = Ig = s
Xoshiro256p(x, y, z)	explicit current/substream/stream states

Functions

rand(rng::Xoshiro256p) -> Float64 Uniform in [0, 1), built from a full 64-bit draw.

rand(rng::Xoshiro256p, T) Supported T: Float64, Float32, Float16, UInt64, and UnitRange{Int64} (uniform, unbiased modulo sampling).

srand!(rng, seed::Vector{UInt64}) -> rng Re-seed an existing generator (first 4 words are used).

reset_stream!(rng) -> rng, reset_substream!(rng) -> rng, next_substream!(rng) -> rng Same semantics as for MRG32k3a; next_substream! performs a short_jump! (leap of $\approx 2^{96}$ values).

short_jump!(rng) -> rng Jump to the beginning of the next 2^{96} -value block (Vigna's short jump).

long_jump!(rng) -> rng Leap $\approx 2^{192}$ values ahead — used to delimit streams.

get_state(rng) -> Vector{UInt64} Copy of the current state Cg.

copy(rng) -> Xoshiro256p Independent copy.

(internal) **next(rng) -> UInt64**: raw 64-bit output; also available as rand(rng, UInt64).

Stream generation

Signature	Description
<code>Xoshiro256plusGen(seed::Vector{UInt64})</code>	create a stream generator (seed must have exactly 4 words)
<code>next_stream!(gen) -> Xoshiro256p</code>	new independent stream (<code>long_jump!</code> from the stored seed)
<code>srand!(gen, seed::Vector{UInt64}) -> gen</code>	reset the stored seed
<code>get_state(gen) -> Vector{UInt64}</code>	seed that will be used by the next <code>next_stream!</code> call

4.4 Xoshiro / xoroshiro families

All variants share one implementation (`LinRNG{N,S}` internally, N = state words, S = scrambler) and expose the same API as [Xoshiro256+](#) above. Exported types:

RNGs	Stream generators
<code>Xoroshiro128p</code> , <code>Xoroshiro128ss</code> , <code>Xoroshiro128pp</code>	<code>Xoroshiro128pGen</code> , <code>Xoroshiro128ssGen</code> , <code>Xoroshiro128ppGen</code>
<code>Xoshiro256p</code> , <code>Xoshiro256ss</code> , <code>Xoshiro256pp</code>	<code>Xoshiro256plusGen</code> , <code>Xoshiro256ssGen</code> , <code>Xoshiro256ppGen</code>
<code>Xoshiro512p</code> , <code>Xoshiro512ss</code> , <code>Xoshiro512pp</code>	<code>Xoshiro512pGen</code> , <code>Xoshiro512ssGen</code> , <code>Xoshiro512ppGen</code>

Constructors accept an `NTuple{N,UInt64}` or a `Vector{<:Unsigned}` of length N (1 or 3 arguments). Available functions: `rand` (`Float64/Float32/Float16`, `UInt64`, ranges), `srand!`, `reset_stream!`, `reset_substream!`, `next_substream!`, `short_jump!`, `long_jump!`, `get_state`, `copy`.

`advance_state!(rng, e::Integer, c::Integer)` -> `rng` Jump forward by n steps ($n = 2^e + c$ if $e > 0$, $n = -2^(-e) + c$ if $e < 0$, $n = c$ if $e = 0$) or backward if $n < 0$. Implemented by computing the jump polynomial $x^n \bmod p(x)$ over $\text{GF}(2)$ (p = the family's characteristic polynomial) and applying it to the current state; distances are reduced modulo the period. Only `Cg` moves — stream/substream boundaries are kept. Cost is $O((64N)^2)$ $\text{GF}(2)$ operations (≈ 10 –500 ms depending on family); for the standard distances prefer `short_jump!/long_jump!`, which use precomputed constants and are allocation-free.

Jump semantics (all xoshiro-family generators):

- `short_jump!(rng)` — jump anchored at the current **substream start** (`Bg`); lands at the next substream boundary (2^{64} values for `xoroshiro128`, 2^{128} for `xoshiro256`, 2^{256} for `xoshiro512`).
- `long_jump!(rng)` — jump anchored at the **stream start** (`Ig`); delimits independent streams.
- `next_stream!(gen)` applies the long-jump polynomial to the stored seed.

All jump constants are the official ones from xoshiro.di.unimi.it and outputs are validated byte-for-byte against the original C implementations.

4.5 Method index

Function	MRG32k3a	Xoshiro256p
rand (Float64)	yes	yes
rand (other floats/integers/bool)	yes	partial
rand (range)	yes	yes (UnitRange{Int64})
reset_stream!	yes	yes
reset_substream!	yes	yes
next_substream!	yes	yes
advance_state!	yes	—
srand!	—	yes
get_state	yes	yes
next_stream! (via Gen)	yes	yes

Part V

Docstrings

Chapter 5

Docstrings

Auto-generated documentation from the package docstrings.

5.1 Abstract types

`RandomDataStreams.AbstractRNGStream` – Type.

`AbstractRNGStream` is an abstraction for creating RNG objects with non-overlapping random numbers.

[source](#)

`RandomDataStreams.AbstractStreamableRNG` – Type.

`AbstractStreamableRNG` is a special kind of RNG where one can move between different stream of number that are known to be non-overlapping.

[source](#)

5.2 MRG32k3a

`RandomDataStreams.MRG32k3a` – Type.

`MRG32k3a()` will generate an instance of `MRG32k3a` with initial seeds `DEFAULT_SEED = [ 12345, 12345, 12345, 12345, 12345, 12345 ]`

`MRG32k3a(x::Vector{Int})` will generate an instance of `MRG32k3a` with initial seeds `x`

`MRG32k3a(x::Vector{Int}, y::Vector{Int}, z::Vector{Int})` will generate an instance of `MRG32k3a` with `Cg = x`, `Bg = y` and `Ig = z`

Examples

```
julia> rng = MRG32k3a();  
  
julia> rand(rng)  
0.12701112204657714
```

[source](#)

`RandomDataStreams.MRG32k3aGen` – Type.

An object that generates independent random number streams.

[source](#)

`RandomDataStreams.checkseed` – Function.

Ensures a given seed is valid for MRG32k3a random number generator. Allocation-free.

[source](#)

5.3 Xoshiro / xoroshiro families

`RandomDataStreams.LinRNG` – Type.

```
LinRNG{N,S} <: AbstractStreamableRNG
```

Generic xoshiro/xoroshiro generator: state of N 64-bit words and scrambler $S \in (:plus, :starstar, :plusplus)$. Holds three checkpoints — current state (Cg), substream start (Bg) and stream start (Ig) — enabling `reset_stream!`, `reset_substream!` and anchored jumps. Use the exported aliases (`Xoroshiro128p`, ..., `Xoshiro512pp`) instead.

[source](#)

`RandomDataStreams.Xoroshiro128p` – Type.

```
Xoroshiro128p <: AbstractStreamableRNG
```

`xoroshiro128+` (Blackman & Vigna): 128-bit state, period $2^{128} - 1$, output $s_0 + s_1$. Fastest small-state generator for floating-point use; the four lower bits have low linear complexity and it has a very mild Hamming-weight dependency after ~ 5 TB of output. Suitable only for mild parallelism (2^{32} streams $\times 2^{64}$ substreams).

[source](#)

`RandomDataStreams.Xoroshiro128ss` – Type.

```
Xoroshiro128ss <: AbstractStreamableRNG
```

`xoroshiro128**` (Blackman & Vigna): 128-bit state, period $2^{128} - 1$, all-purpose scrambler (`rotl(s1*5,7)*9`), 1-dimensionally equidistributed. Suitable only for mild parallelism (2^{32} streams $\times 2^{64}$ substreams).

[source](#)

`RandomDataStreams.Xoroshiro128pp` – Type.

```
Xoroshiro128pp <: AbstractStreamableRNG
```

`xoroshiro128++` (Blackman & Vigna): 128-bit state, period $2^{128} - 1$, all-purpose scrambler (`rotl(s0+s1,17)+s0`). Same parallelism limits as the other 128-bit variants.

[source](#)

RandomDataStreams.Xoshiro256p – Type.

```
Xoshiro256p <: AbstractStreamableRNG
```

xoshiro256+ (Blackman & Vigna): 256-bit state, period $2^{256} - 1$, output $s_0 + s_3$. Slightly faster than $**/+$ when generating only floating-point numbers from the upper bits; avoid for raw 64-bit consumption.

[source](#)

RandomDataStreams.Xoshiro256ss – Type.

```
Xoshiro256ss <: AbstractStreamableRNG
```

xoshiro256** (Blackman & Vigna): 256-bit state, period $2^{256} - 1$, all-purpose, 3-dimensionally equidistributed (default PRNG of Lua and .NET 6).

[source](#)

RandomDataStreams.Xoshiro256pp – Type.

```
Xoshiro256pp <: AbstractStreamableRNG
```

xoshiro256++ (Blackman & Vigna): 256-bit state, period $2^{256} - 1$, all-purpose, 3-dimensionally equidistributed (default PRNG of Julia and Rust's SmallRng). The recommended general-purpose variant.

Examples

```
julia> rng = Xoshiro256pp(fill(UInt64(42), 4));  
  
julia> rand(rng, UInt64)  
0x000000002a00002a
```

[source](#)

RandomDataStreams.Xoshiro512p – Type.

```
Xoshiro512p <: AbstractStreamableRNG
```

xoshiro512+ (Blackman & Vigna): 512-bit state, period $2^{512} - 1$, output $s_0 + s_2$. Floating-point oriented; same caveats as Xoshiro256p.

[source](#)

RandomDataStreams.Xoshiro512ss – Type.

```
Xoshiro512ss <: AbstractStreamableRNG
```

xoshiro512** (Blackman & Vigna): 512-bit state, period $2^{512} - 1$, all-purpose, 7-dimensionally equidistributed.

[source](#)

RandomDataStreams.Xoshiro512pp – Type.

```
Xoshiro512pp <: AbstractStreamableRNG
```

xoshiro512++ (Blackman & Vigna): 512-bit state, period $2^{512} - 1$, all-purpose. Offers 2^{256} substreams of 2^{256} values — more than any application needs; prefer Xoshiro256pp unless you have a specific reason.

[source](#)

Stream generators

RandomDataStreams.LinGen – Type.

```
LinGen{N,S} <: AbstractRNGStream
```

Stream generator producing non-overlapping streams of `LinRNG{N,S}`: each call to `next_stream!` applies the family's long jump to the stored seed. Use the exported aliases (`Xoroshiro128pGen`, ..., `Xoshiro512ppGen`) instead.

[source](#)

RandomDataStreams.Xoroshiro128pGen – Type.

```
Xoroshiro128pGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoroshiro128p streams via a long jump (2^{96} values) per call. Seeds are 2 UInt64 words.

[source](#)

RandomDataStreams.Xoroshiro128ssGen – Type.

```
Xoroshiro128ssGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoroshiro128ss streams via a long jump (2^{96} values) per call. Seeds are 2 UInt64 words.

[source](#)

RandomDataStreams.Xoroshiro128ppGen – Type.

```
Xoroshiro128ppGen <: AbstractRNGStream
```


Stream generator minting non-overlapping Xoroshiro128pp streams via a long jump (2^{96} values) per call. Seeds are 2 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro256plusGen – Type.

```
Xoshiro256plusGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256p streams via a long jump (2^{192} values) per call. Seeds are 4 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro256ssGen – Type.

```
Xoshiro256ssGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256ss streams via a long jump (2^{192} values) per call. Seeds are 4 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro256ppGen – Type.

```
Xoshiro256ppGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro256pp streams via a long jump (2^{192} values) per call. Seeds are 4 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro512pGen – Type.

```
Xoshiro512pGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro512p streams via a long jump (2^{384} values) per call. Seeds are 8 UInt64 words.

[source](#)

RandomDataStreams.Xoshiro512ssGen – Type.

```
Xoshiro512ssGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro512ss streams via a long jump (2^{384} values) per call. Seeds are 8 UInt64 words.

[source](#)

`RandomDataStreams.Xoshiro512ppGen` – Type.

```
Xoshiro512ppGen <: AbstractRNGStream
```

Stream generator minting non-overlapping Xoshiro512pp streams via a long jump (2^{384} values) per call. Seeds are 8 UInt64 words.

[source](#)

Functions

`Base.rand` – Method.

Produces a raw random number with 32 bits of precision.

[source](#)

`Base.rand` – Method.

Return a random Float64 in [0, 1).

[source](#)

`Base.rand` – Method.

Generates a Float32 from any xoshiro/xoroshiro generator.

[source](#)

`Base.rand` – Method.

Generates a Float16 from any xoshiro/xoroshiro generator.

[source](#)

`Base.rand` – Method.

Generates an Int64 uniformly distributed in the given range.

[source](#)

`Base.rand` – Method.

Hook required by Random's generic machinery for Float64-native generators: produces a Float64 in [1, 2). Everything else (ints, arrays, shuffle...) derives from this without further plumbing.

[source](#)

`RandomDataStreams.short_jump!` – Function.

Jumps forward by 2^{96} values (xoroshiro128), 2^{128} (xoshiro256) or 2^{256} (xoshiro512): the start of the next non-overlapping substream.

[source](#)

`RandomDataStreams.long_jump!` – Function.

Jumps forward by 2^{96} (xoroshiro128), 2^{192} (xoshiro256) or 2^{384} (xoshiro512) values: the start of the next independent stream.

[source](#)

`RandomDataStreams.advance_state!` – Function.

Given a random number generator, jumps n steps forward if $n > 0$ (or $-n$ steps backwards if $n < 0$), where if $e > 0$, let $n = 2^e + c$; if $e < 0$, let $n = -2^{(-e)} + c$; if $e = 0$, let $n = c$.

Examples

```
julia> rng = MRG32k3a();

julia> advance_state!(rng, Int64(2), Int64(-1)); # skip n = 2^2 - 1 = 3 values

julia> rand(rng)
0.8258468629271136
```

source

Given a random number generator, jumps n steps forward if $n > 0$ (or $-n$ steps backwards if $n < 0$), where if $e > 0$, let $n = 2^e + c$; if $e < 0$, let $n = -2^{(-e)} + c$; if $e = 0$, let $n = c$.

The distance is reduced modulo the period ($2^{(64N)} - 1$), so any magnitude is accepted. Only the current position moves; the stream/substream boundaries (B_g , I_g) are left untouched. Costs $O((64N)^2)$ GF(2) operations.

source

`RandomDataStreams.srand!` – Function.

Seeds a generator with the first words of seed.

source

```
srand!(gen, seed::Vector{UInt64}) -> gen
```

Resets the seed that the next `next_stream!` call will use.

source

`RandomDataStreams.get_state` – Function.

`get_state` return the seed used to generate non-overlapping MRGs

source

```
get_state(rng::LinRNG) -> Vector{UInt64}
```

Copy of the current state (C_g); restore it into a fresh generator via its three-argument constructor.

source

```
get_state(gen) -> Vector{UInt64}
```

Seed that will be used by the next `next_stream!` call.

source

`RandomDataStreams.next_stream!` – Function.

Given an RNG generator object, returns the next RNG stream.

[source](#)

Given an RNG generator object, returns the next RNG stream.

[source](#)

`RandomDataStreams.reset_stream!` – Function.

Resets a given random number generator to the beginning of the current stream.

[source](#)

```
reset_stream!(rng) -> rng
```

Rewind the generator to the very beginning of its current stream ($Cg = Bg = Ig$).

[source](#)

`RandomDataStreams.reset_substream!` – Function.

Resets a given random number generator to the beginning of the current substream.

[source](#)

```
reset_substream!(rng) -> rng
```

Rewind the generator to the beginning of its current substream ($Cg = Bg$).

[source](#)

`RandomDataStreams.next_substream!` – Function.

Takes a random number generator and shifts seed to next substream.

[source](#)

```
next_substream!(rng) -> rng
```

Move the generator to the start of the next substream (anchored jump from Bg): 2^{64} values for xoroshiro128, 2^{128} for xoshiro256, 2^{256} for xoshiro512.

[source](#)

Part VI

Implementation Notes

Chapter 6

Implementation Notes

6.1 MRG32k3a

MRG32k3a is L'Ecuyer's combined multiple recursive generator of order 3. It maintains two 3-component integer states updated by the recurrences

```
p1 = (1403580 · x2 - 810728 · x1) mod m1,    m1 = 232 - 209
p2 = (527612 · y2 - 1370589 · y0) mod m2,    m2 = 232 - 22853
u = (p1 - p2) / m1    (with wrap-around) ∈ [0, 1)
```

The combined generator has period $\approx 2^{191}$.

Modular arithmetic without overflow

All state components stay below $m1 < 2^{32}$, so products fit in Float64 exactly ($< 2^{53}$). The helper `MultModM(a, s, c, m)` computes $(a \cdot s + c) \bmod m$ using this fact and falls back to a split-by- 2^{17} computation when intermediate values would exceed 2^{53} . Matrix helpers:

- `MatVecModM(A, s, m)` — matrix-vector product mod m
- `MatMatModM(A, B, m)` — matrix product mod m
- `MatTwoPowModM(A, e, m)` — A raised to the power 2^e
- `MatPowModM(A, n, m)` — A^n by binary exponentiation

Streams, substreams, jumps

Moving a stream forward by k steps is a linear operation on its state, hence a precomputed matrix power applied to the state vector:

Operation	Matrix	Step
<code>next_stream!(gen)</code>	<code>A1p127, A2p127</code>	2^{127} values
<code>next_substream!(rng)</code>	<code>A1p76, A2p76</code>	2^{76} values
<code>advance_state!(rng, e, c)</code>	<code>MatTwoPowModM / InvA1, InvA2</code>	arbitrary (inverse matrices allow backward jumps)

The RNG keeps three checkpoints: C_g (current), B_g (substream start), I_g (stream start), which is what makes `reset_substream!` and `reset_stream!` $O(1)$.

6.2 Xoshiro256+

xoshiro256+ (Blackman & Vigna, 2019) keeps four 64-bit words and produces one word per step with only shifts, XORs and rotations:

```
result = s1 + s4
t = s2 << 17
s3 = xor(s3, s1); s4 = xor(s4, s2); s2 = xor(s2, s3); s1 = xor(s1, s4); s3 = xor(s3, t); s4 =
↪ rotl(s4, 45)
```

The output word is the sum of two internal words (+ variant): fastest of the xoshiro family for float generation, though the low three bits of the raw output have limited linear complexity — irrelevant once scaled to `Float64` (53 bits used). Period: $2^{256} - 1$.

Jumps

`short_jump!` and `long_jump!` use Vigna's published jump polynomials. They are computed as XOR-linear combinations of states visited while stepping through the 256 bits of each jump constant. `RandomDataStreams.jl` hoists the jump constants into compile-time tuples and evaluates jumps allocation-free on immutable state tuples.

`RandomDataStreams` semantics differ subtly from the reference C code: jumps are **anchored** at stream boundaries. `short_jump!(rng)` first resets $C_g \leftarrow B_g$, then applies the jump polynomial, and stores the result in both C_g and B_g ; `long_jump!(rng)` does the same with respect to I_g . This matches L'Ecuyer's stream model (`next_substream!/reset_stream!`) and makes scenario replay reproducible regardless of consumption inside the current substream.

Families and scramblers

The package implements all 64-bit variants from `xoshiro.di.unimi.it` through a single generic type `LinRNG{N,S}` (state words $\times$ scrambler):

- transition `_lin_step` for `NTuple{2}` (xoroshiro128: $a=24, b=16, c=37$), `NTuple{4}` (xoshiro256) and `NTuple{8}` (xoshiro512);
- scramblers `+`, `** (rotr(s2·5,7)·9)` and `++ (rotr(s1+s4,23)+s1)` for 256; family-specific constants);
- per-family jump constants shared by all scramblers.

Exported aliases: `Xoroshiro128p/ss/pp`, `Xoshiro256p/ss/pp`, `Xoshiro512p/ss/pp` plus matching `*Gen` stream generators.

All nine variants are regression-tested against byte-exact outputs produced by the original C implementations compiled with `gcc` (sequences, short jumps and long jumps).

Arbitrary forward/backward jumps

Because the transition is multiplication by x in $\text{GF}(2)[x]/p(x)$ — with p the family's characteristic polynomial, as published in Vigna's reference files — one can jump any distance n in constant time by applying the polynomial $x^n \bmod p$ through the same accumulate-and-step loop as the fixed jumps. Backward distances use the multiplicative order of x : $x^{(-n)} = x^{(2^{\deg}-1-n)}$ ($\deg = 64N$). `RandomDataStreams.jl` implements a small $\text{GF}(2)$ polynomial engine (`_poly_mul_mod`, `_poly_pow_x`, `BigInt` exponents reduced modulo the period) and exposes it as `advance_state!(rng, e, c)` with the exact distance convention of MRG32k3a. Correctness is property-tested: fixed jumps coincide with `short_jump!`, round trips ($+k$ then $-k$) restore the state bit-for-bit, and backward-jumped generators re-emit previously seen values.

State representation

The state lives in `NTuple{4, UInt64}` fields. Immutable tuples let the JIT keep the whole working set in registers inside next, eliminating bounds checks and heap traffic that a `Vector{UInt64}` representation incurs. Measured throughput: $\approx 10^9$ draws/s on a single core.

6.3 Integer outputs of MRG32k3a

MRG32k3a natively yields ~ 31 bits per draw (the difference $p1 - p2$). Wider unsigned types are assembled from 16-bit chunks of that value; narrower types are truncations. Consequence: these integers are *not* uniform over their full width — fine for indexing/shuffling/flags in simulations, not for cryptographic use.

6.4 Testing strategy

Reference sequences were captured from the reference implementations (L'Ecuyer's C code semantics for MRG32k3a; Vigna's constants for the xoshiro jumps) and regression-checked after optimization: identical outputs are produced for every supported type, plus reset/jump/state round-trips.

6.5 Performance snapshot

Single core, Julia 1.12 (indicative):

Operation	Throughput
<code>rand(::MRG32k3a)</code>	$\approx 200 \times 10^6/\text{s}$
<code>rand(::Xoroshiro128p)</code>	$\approx 1200 \times 10^6/\text{s}$
<code>rand(::Xoshiro256p/ss/pp)</code>	$\approx 1340 \times 10^6/\text{s}$
<code>rand(::Xoshiro512p/ss/pp)</code>	$\approx 1150 \times 10^6/\text{s}$
<code>short_jump!</code> / <code>long_jump!</code>	0 allocations

Part VII

Generator Comparison

Chapter 7

Comparing generators: xoshiro variants vs MRG32k3a

RandomDataStreams.jl lets you pick between two very different generator designs. This page helps you choose.

7.1 Side-by-side

Property	MRG32k3a	Xoroshiro128* (p/ss/pp)	Xoshiro256* (p/ss/pp)	Xoshiro512* (p/ss/pp)
State (bits)	192 (6 × Int64)	128	256	512
Period	$\approx 2^{191}$	$2^{128} - 1$	$2^{256} - 1$	$2^{512} - 1$
Native output	Float64 [0,1), ~32-bit resolution	UInt64	UInt64	UInt64
Throughput (this package, single core)	≈ 200 M/s	≈ 1200 M/s	≈ 1340 M/s	≈ 1150 M/s
Stream mechanism	matrix power $A^{2^{127}}$ on the seed	long_jump! polynomial (2^{96})	long_jump! (2^{192})	long_jump! (2^{384})
Substream mechanism	matrix power $A^{2^{76}}$	short_jump! (2^{64})	short_jump! (2^{128})	short_jump! (2^{256})
Backward jumps	yes (advance_state!)	yes (advance_state!, GF(2) polynomials)	yes (advance_state!)	yes (advance_state!)
Arbitrary-jump cost	O(e) matrix products	O(deg ²) GF(2) ops (~10-500 ms)	same	same
Equidistribution	well analysed theory	1-dim (**/++) / 0-dim (+)	3-dim (**/++) / 2-dim (+)	7-dim (**/++) / 6-dim (+)
BigCrush (Vigna's shootout)	passes	passes	passes	passes
Parallelism scale advised	large	mild ($\approx 2^{32}$ streams)	very large	extreme

(* means any of the +, **, ++ scramblers.)

7.2 When to choose MRG32k3a

- You need **provably non-overlapping streams with published, peer-reviewed parameters** (L'Ecuyer et al. 2002), e.g. to stay compatible with simulation libraries built on that model (SSJ, Simio, Arena-style stream ids...).
- Your application consumes **floats only**: MRG32k3a emits them natively.
- You can afford $\sim 6\times$ slower generation.

Note: backward jumping is *no longer* a differentiator — every RandomDataStreams generator supports `advance_state!(rng, e, c)` with negative distances.

7.3 When to choose a xoshiro/xoroshiro variant

- You want **maximum throughput** (all are $\approx 6\text{--}7\times$ faster than MRG32k3a here) or raw 64-bit integers.
- Choose your state size by parallelism needs:
 - Xoroshiro128*: embedded/GPU or few workers only — its 2^{64} substreams per 2^{32} streams are enough for mild parallelism. Note Xoroshiro128p has a mild Hamming-weight dependency after ~ 5 TB of output; prefer ss/pp.
 - Xoshiro256*: the sweet spot for general use (Julia itself uses xoshiro256++ as its default RNG). Use `+` if you generate *only* floating-point numbers from the high bits ($\sim 15\%$ faster historically); otherwise prefer `++/**`, whose outputs are fully equidistributed.
 - Xoshiro512*: essentially never needed for period reasons — any period $\geq 2^{256}$ is beyond every imaginable need — but handy when you want many more independent substreams than 2^{128} without changing design.

7.4 Scrambler choice within a family

Scram- bler	Output	Best for
<code>+</code>	sum of two state words	float-only workloads using the top bits; lowest 3 bits have low linear complexity
<code>**</code>	<code>rotl(s2·5, 7)·9</code>	all-purpose; maximal equidistribution (e.g. .NET/Lua default)
<code>++</code>	<code>rotl(s1+s4, k)+s1</code>	all-purpose; Rust's <code>SmallRng</code> , Julia's default

All three share the same transition and therefore the same jump constants: streams/substreams behave identically across scramblers of one family.

7.5 Practical notes

- Seeding: seed states must not be all-zero. For hash-like seeds, initialize with `splitmix64` outputs (see Vigna's recommendation).
- The package's jumps are **anchored at stream boundaries** ($Cg \leftarrow Bg \leftarrow Ig$), matching the L'Ecuyer stream model: `next_substream!` always lands at the same place regardless of how far you consumed in the current substream, which makes scenario replay reproducible.

Part VIII

FAQ

Chapter 8

FAQ

8.1 Why is an all-zero state forbidden?

For the xoshiro/xoroshiro families the all-zero state maps to itself: the generator would output only zeros forever. MRG32k3a's two components have the same degeneracy (an all-zero component stays zero). Constructors and `checkseed` reject such seeds; `Random.seed!(rng, ::Integer)` never produces one.

8.2 Are MRG32k3a integer outputs uniform over their full width?

No. MRG32k3a natively produces ~ 31 bits per draw ($p1 - p2$); wider integers are assembled from 16-bit chunks of that value. They are fine for indices, shuffling, flags and acceptance tests in simulations — not uniform over $2^{64}/2^{128}$. Use a xoshiro variant when you need full-width raw words.

8.3 Why doesn't `sample(v, k)` work with `RandomDataStreams` generators?

`sample` comes from `StatsBase.jl`, not from the `Random` standard library — even Julia's own RNGs do not provide it without that package. Everything in `Random.jl` works with `RandomDataStreams` generators.

8.4 Where did `srand`, `next_stream`, `short_jump`, `long_jump` go?

They mutated their argument despite not ending in `!`; they are deprecated in favour of:

Deprecated	Replacement
<code>srand(rng/gen, seed)</code>	<code>srand!(rng/gen, seed)</code>
<code>next_stream(gen)</code>	<code>next_stream!(gen)</code>
<code>short_jump(rng)</code>	<code>short_jump!(rng)</code>
<code>long_jump(rng)</code>	<code>long_jump!(rng)</code>

The old names still work but emit a deprecation warning.

8.5 How do I run truly parallel simulations?

Never share one generator object across tasks/workers. Give each worker its own stream *starting seed* (see [Streams & Substreams](#)); streams produced by successive `next_stream!` calls are guaranteed non-overlapping.

8.6 Which generator should I pick?

See [Generator Comparison](#). Short answer: Xoshiro256pp for general use, MRG32k3a for L'Ecuyer-compatible stream semantics.